

Project (2) Report

Title:

Cryptography & Password Hashing

Type:

Applied Security Implementation

Submitted By:

ALIZA JAVED

University of Engineering and Technology, Lahore

BS Computer Science

CX-2026-CYBER

1. Introduction

This project demonstrates the implementation of secure authentication techniques using Python and modern cryptographic libraries. It covers password hashing with **bcrypt**, password verification, **SHA-256** hashing, message encryption and decryption using **Fernet**, and a basic user authentication system. The project highlights the importance of protecting user credentials and sensitive information by applying secure authentication and encryption practices commonly used in modern applications.

2. Objectives

The main objectives of this project are:

- Implement secure password hashing using **bcrypt**.
- Verify user passwords securely without storing plain-text passwords.
- Generate **SHA-256** hashes for data integrity demonstration.
- Encrypt and decrypt sensitive data using **Fernet** encryption.
- Develop a secure user authentication system with registration and login.
- Demonstrate the importance of secure authentication practices in modern applications.

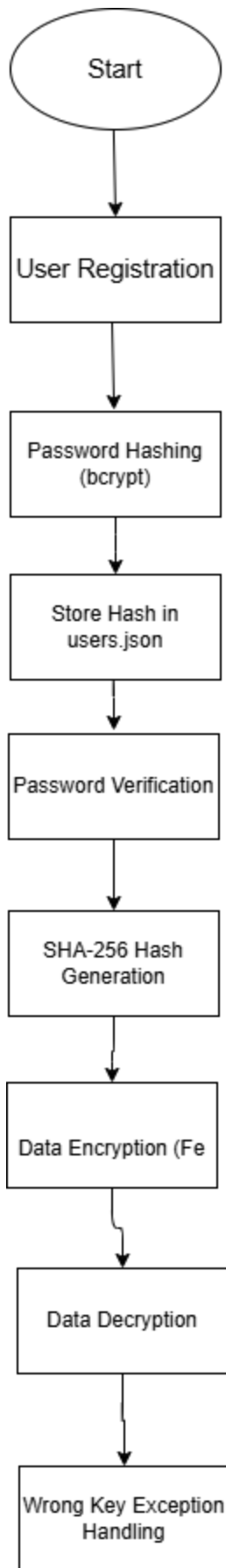
3. Tools & Technologies Used

Tool / Technology	Purpose
Python 3	Programming language used to develop the authentication system.
bcrypt	Securely hashes and verifies user passwords.
hashlib (SHA-256)	Generates SHA-256 hashes for data integrity demonstration.
cryptography (Fernet)	Encrypts and decrypts sensitive data using a secret key.

JSON	Stores user credentials (hashed passwords) in a local file.
OS Module	Checks file existence and manages file operations.
Visual Studio Code	Code editor used for development.
Kali Linux	Operating system used to implement and test the project.

4. Project Workflow (Methodology)

The project was completed using the following workflow:



Methodology

1. User credentials are entered through the registration process.
2. Passwords are securely hashed using the **bcrypt** library before storage.
3. The hashed passwords are saved in a **JSON** file.
4. During login, the entered password is verified against the stored hash.
5. A **SHA-256** hash is generated to demonstrate cryptographic hashing.
6. Sensitive data is encrypted using **Fernet** with a generated secret key.
7. The encrypted data is decrypted using the correct secret key, while an incorrect key raises an exception.

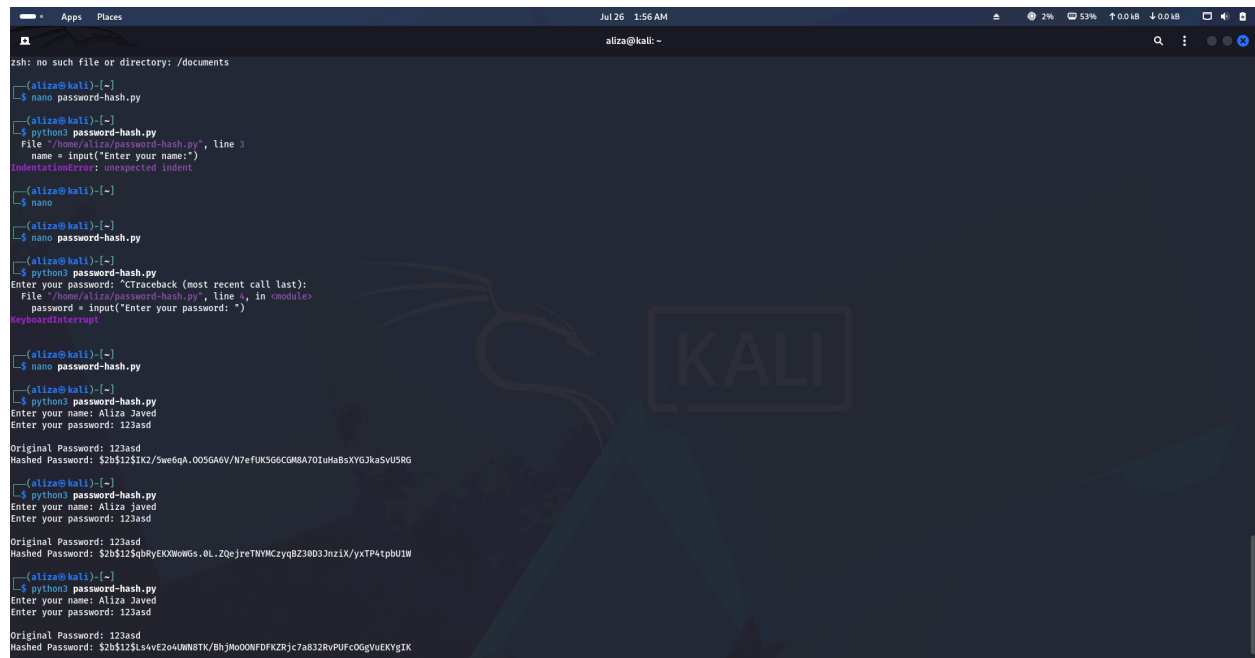
5. Project Implementation

5.1 Password Hashing (bcrypt)

Password hashing was implemented using the **bcrypt** library to securely protect user passwords. Instead of storing plain-text passwords, the application stores only the hashed values, making them difficult to recover even if the database is compromised.

Output: Password successfully converted into a bcrypt hash.

Screenshot:



```
zsh: no such file or directory: /documents
(aliza@kali)-[~]
└─$ nano password-hash.py
(aliza@kali)-[~]
└─$ python3 password-hash.py
File "/home/aliza/password-hash.py", line 3
  name = input("Enter your name:")
        ^
IndentationError: unexpected indent
(aliza@kali)-[~]
└─$ nano
(aliza@kali)-[~]
└─$ nano password-hash.py
(aliza@kali)-[~]
└─$ python3 password-hash.py
Enter your password: "Ctrlceback (most recent call last):
File "/home/aliza/password-hash.py", line 4, in <module>
  password = input("Enter your password: ")
KeyboardInterrupt
(aliza@kali)-[~]
└─$ nano password-hash.py
(aliza@kali)-[~]
└─$ python3 password-hash.py
Enter your name: Aliza Javed
Enter your password: 123asd
Original Password: 123asd
Hashed Password: $2b$12$1KZ/5we6GA.005GA0V/N7eFUK5G6CG8A70IuHaBsXYGJkaSVU5RG
(aliza@kali)-[~]
└─$ python3 password-hash.py
Enter your name: Aliza Javed
Enter your password: 123asd
Original Password: 123asd
Hashed Password: $2b$12$g8yEK0W0G5.0L..ZqeJreTNVNCzyqBZ803Jmz1X/yxTP4tpbU1W
(aliza@kali)-[~]
└─$ python3 password-hash.py
Enter your name: Aliza Javed
Enter your password: 123asd
Original Password: 123asd
Hashed Password: $2b$12$1s4vE2o4UWN8TK/BhjMo0DNFDfKZBjc7a832RvPUFcoGvUeKYg1K
```

Observations:

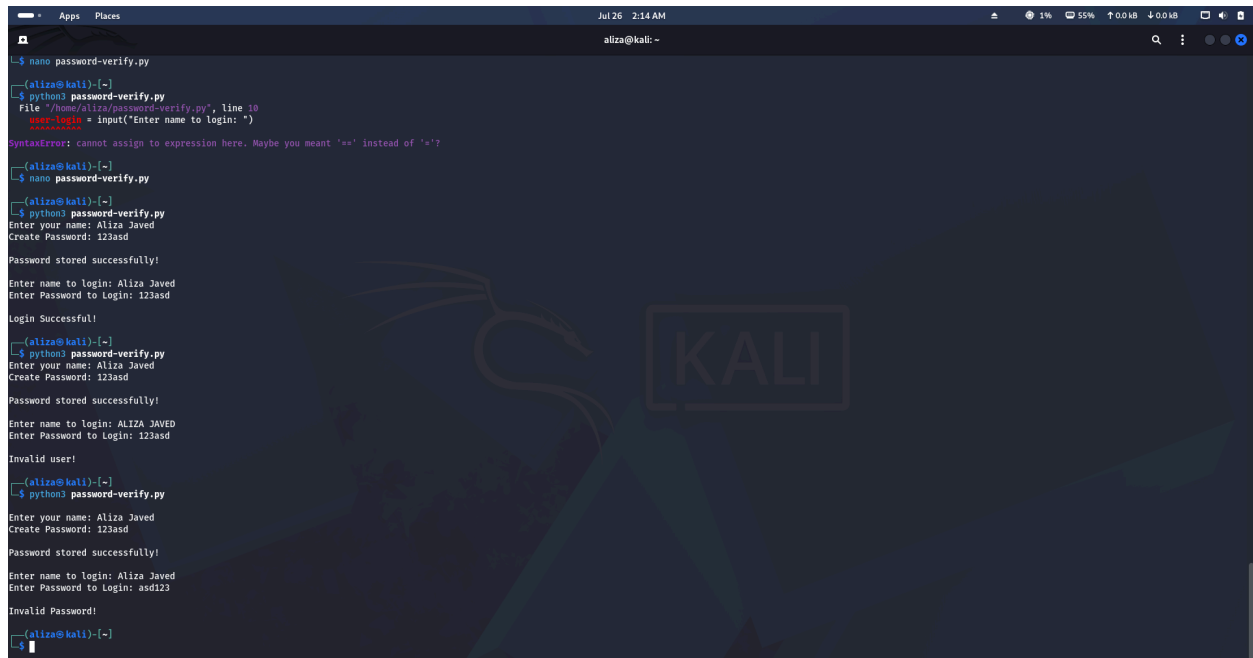
I executed the **password_hash.py** program three times using the same password. Each execution generated a different hash value because **bcrypt** uses a unique salt for every hashing operation, making password storage more secure.

5.2 Password Verification

The stored bcrypt hash was verified using the entered password. The system grants access only when the entered password matches the stored hash; otherwise, it displays an invalid password message.

Output: Login successful for valid credentials and invalid password for incorrect credentials.

Screenshot:



```
alisha@kali: ~$ nano password-verify.py
alisha@kali: ~$ python3 password-verify.py
File ~/alisha/password-verify.py, line 10
user_login = input("Enter name to login: ")
^^^^^^^^
SyntaxError: cannot assign to expression here. Maybe you meant '==' instead of '='?

alisha@kali: ~$ nano password-verify.py
alisha@kali: ~$ python3 password-verify.py
Enter your name: Aliza Javed
Create Password: 123asd
Password stored successfully!
Enter name to login: Aliza Javed
Enter Password to Login: 123asd
Login Successful!

alisha@kali: ~$ python3 password-verify.py
Enter your name: Aliza Javed
Create Password: 123asd
Password stored successfully!
Enter name to login: ALIZA JAVED
Enter Password to Login: 123asd
Invalid user!

alisha@kali: ~$ python3 password-verify.py
Enter your name: Aliza Javed
Create Password: 123asd
Password stored successfully!
Enter name to login: Aliza Javed
Enter Password to Login: asd123
Invalid Password!

alisha@kali: ~$
```

Observations:

I executed the **password-verify.py** program multiple times to test different authentication scenarios:

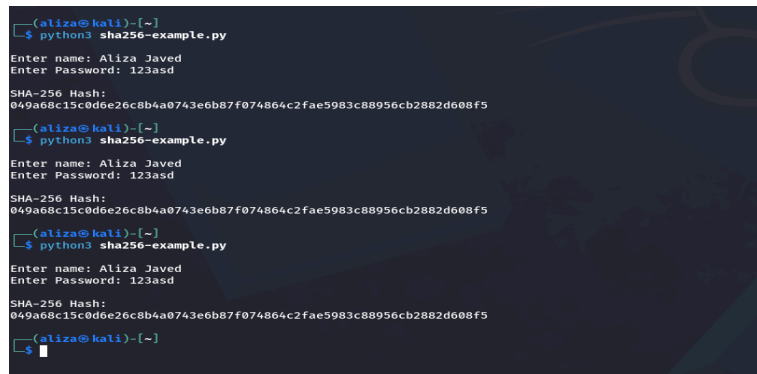
- **Case 1:** The user entered a valid username and password, Program verify the password and the login was successful.
- **Case 2:** The user entered an invalid username with a valid password, and the login failed.
- **Case 3:** The user entered a valid username with an invalid password, and the login failed.
- **Case 4:** Both the username and password were invalid, and the login failed.

5.3 SHA-256 Hashing

The **hashlib** library was used to generate SHA-256 hashes. This demonstrates how the same input always produces the same hash, while different inputs generate completely different hash values.

Output: SHA-256 hash generated successfully.

Screenshot:



```
(aliza@kali)-[~]
$ python3 sha256-example.py
Enter name: Aliza Javed
Enter Password: 123asd
SHA-256 Hash:
049a68c15c0d6e26c8b4a0743e6b87f074864c2fae5983c88956cb2882d608f5

(aliza@kali)-[~]
$ python3 sha256-example.py
Enter name: Aliza Javed
Enter Password: 123asd
SHA-256 Hash:
049a68c15c0d6e26c8b4a0743e6b87f074864c2fae5983c88956cb2882d608f5

(aliza@kali)-[~]
$ python3 sha256-example.py
Enter name: Aliza Javed
Enter Password: 123asd
SHA-256 Hash:
049a68c15c0d6e26c8b4a0743e6b87f074864c2fae5983c88956cb2882d608f5

(aliza@kali)-[~]
$
```

Observations:

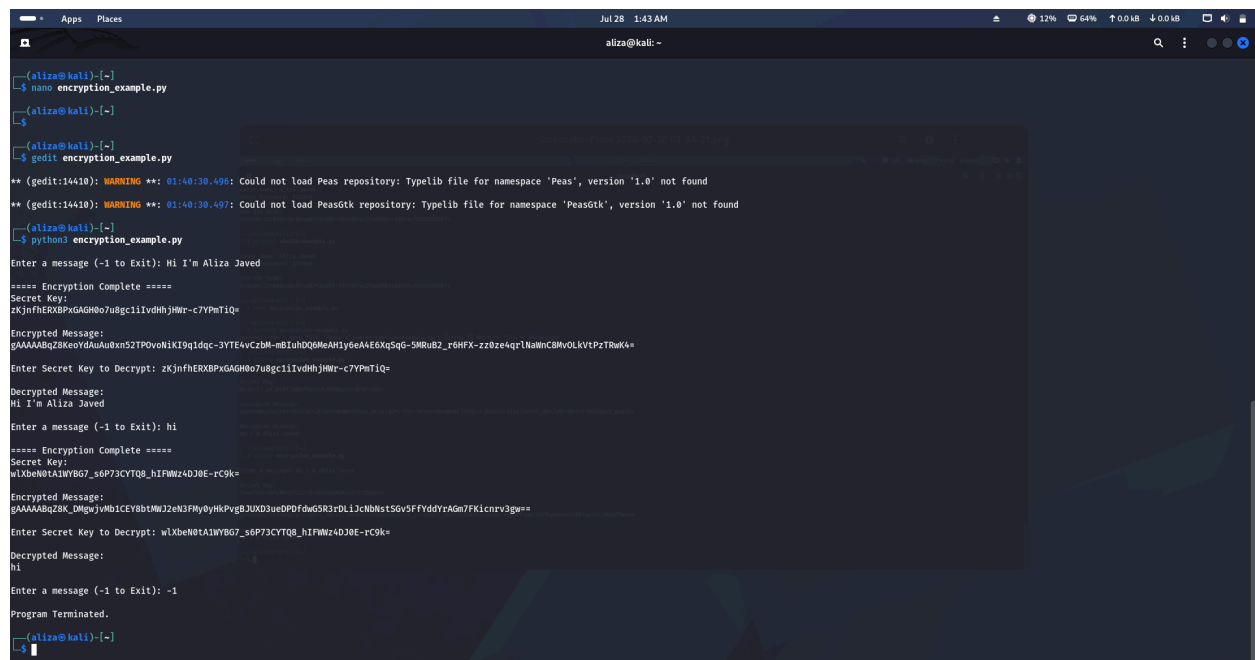
I executed the **sha256-example.py** program three times using the same password. Each execution generated the **same SHA-256 hash**, demonstrating that SHA-256 is a deterministic hashing algorithm, meaning the same input always produces the same output.

5.4 Data Encryption & Decryption

The **Fernet** module from the **cryptography** library was used to encrypt and decrypt messages. A unique secret key is generated for encryption, and the same key is required to recover the original message. Entering an incorrect key results in a decryption failure. As user enter the data program generate secret key and use to encrypt that data , Then program will ask the user to enter the same secret key that was generated and Now it will use to decrypt the same data that was entered by user

Output: Message encrypted and successfully decrypted using the correct secret key.

Screenshot:



```
alisha@kali:~$ nano encryption_example.py
alisha@kali:~$ python3 encryption_example.py
Enter a message (-1 to Exit): Hi I'm Aliza Javed
===== Encryption Complete =====
Secret Key:
ZKjnfHERXBPxGAGH0u7u8gc1iIvdhJHmr-c7YPmTiQ=
Encrypted Message:
gAAAAABqZ8keYdAuAu52TP0voNiK1D91dqc-3YTE4vCzBm-m8IuhDQ6MeAHjy6A4E6XqSgQ-SMRu82_r0HFX-zz0ze4qLNameCNvOLKvPzTrwK4=
Enter Secret Key to Decrypt: ZKjnfHERXBPxGAGH0u7u8gc1iIvdhJHmr-c7YPmTiQ=
Decrypted Message:
Hi I'm Aliza Javed
Enter a message (-1 to Exit): hi
===== Encryption Complete =====
Secret Key:
wLXbeN0A1WYB07_s6P73CYT08_hIFWmZ4D30E-rC9k=
Encrypted Message:
gAAAAABqZ8kDmgpJmB1CEY8bt1Mw22eN3FMy0yHkPvgBJUXD3ueDPDfdwG5R3rDLi3cNBnStSGv5FFYddYrA6m7FK1cnrv3ge=
Enter Secret Key to Decrypt: wLXbeN0A1WYB07_s6P73CYT08_hIFWmZ4D30E-rC9k=
Decrypted Message:
hi
Enter a message (-1 to Exit): -1
Program Terminated.
alisha@kali:~$
```

Observations:

I executed the **encryption_example.py** program to demonstrate secure data encryption and decryption. The user first enters a message, after which the program generates a unique secret key and encrypts the message using the **Fernet** algorithm. The encrypted message and the generated secret key are displayed. The program then prompts the user to enter the secret key to decrypt the message. If the correct key is entered, the original message is successfully recovered; otherwise, decryption fails with an error. After completing the process, the program asks the user to enter another message or type **-1** to terminate the program.

5.5 Secure Authentication System

A secure authentication system was developed that allows users to register and log in. During registration, passwords are securely hashed using **bcrypt** and stored in a JSON file. During login, the entered password is verified against the stored hash. Only users with valid credentials are successfully authenticated.

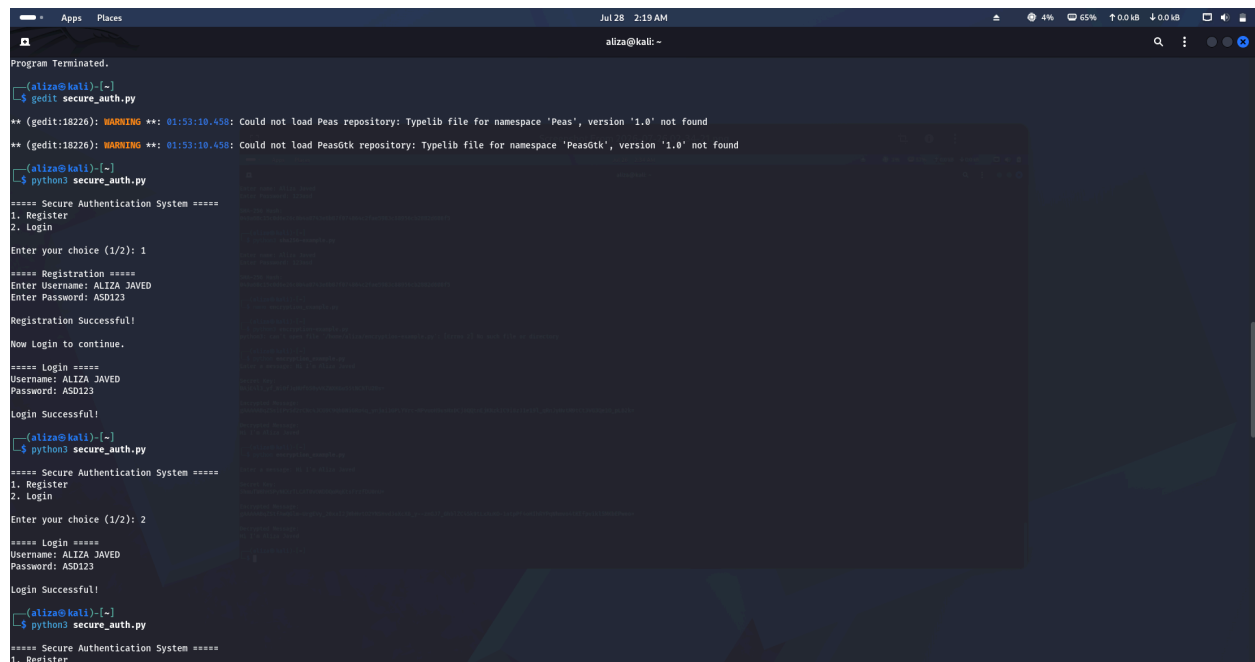
Case 1: If a user is already registered and chooses the **Login** option, the user can log in successfully by entering the correct credentials. After successful authentication, the program terminates.

Case 2: If a new user selects the **Register** option, the account is created successfully. The program then prompts the user to log in using the same credentials entered during registration. If the correct credentials are provided, the login is successful; otherwise, authentication fails.

Features:

- User Registration
- User Login
- Secure Password Hashing
- Password Verification
- JSON-based User Storage

Screenshot & Observation:



```
Program Terminated.
(aliza@kali)-[~]
└─$ gedit secure_auth.py

** (gedit:18226): WARNING **: 01:53:10.458: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:18226): WARNING **: 01:53:10.458: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(aliza@kali)-[~]
└─$ python3 secure_auth.py

===== Secure Authentication System =====
1. Register
2. Login

Enter your choice (1/2): 1

===== Registration =====
Enter Username: ALIZA_JAVED
Enter Password: ASD123

Registration Successful!

Now Login to continue.

===== Login =====
Username: ALIZA_JAVED
Password: ASD123

Login Successful!

(aliza@kali)-[~]
└─$ python3 secure_auth.py

===== Secure Authentication System =====
1. Register
2. Login

Enter your choice (1/2): 2

===== Login =====
Username: ALIZA_JAVED
Password: ASD123

Login Successful!

(aliza@kali)-[~]
└─$ python3 secure_auth.py

===== Secure Authentication System =====
1. Register
```

I tested the **secure_auth.py** program using multiple authentication scenarios to verify its functionality.

Execution 1: I registered a new user and then logged in using the same credentials. The authentication was successful.

```
alisha@kali: ~$ python3 secure_auth.py
===== Secure Authentication System =====
1. Register
2. Login
Enter your choice (1/2): 2
===== Login =====
Username: Aliza Javed
Password: 123asd
Invalid Username!
alisha@kali: ~$ python3 secure_auth.py
===== Secure Authentication System =====
1. Register
2. Login
Enter your choice (1/2): 2
===== Login =====
Username: Aliza Javed
Password: asd123
Invalid Username!
alisha@kali: ~$ python3 secure_auth.py
===== Secure Authentication System =====
1. Register
2. Login
Enter your choice (1/2): 2
===== Login =====
Username: ALIZA JAVED
Password: asd123
Invalid Password!
alisha@kali: ~$ python3 secure_auth.py
===== Secure Authentication System =====
1. Register
2. Login
Enter your choice (1/2): 1
```

Execution 2: I tested two invalid login scenarios. In the first run, I entered an **invalid username** with the **correct password**, and the login failed. In the second run, I entered the **correct username** with an **invalid password**, and the login also failed.

```
alisha@kali: ~$ python3 secure_auth.py
1. Register
2. Login
Enter your choice (1/2): 1
===== Registration =====
Enter Username: ALI
Enter Password: asd123
Registration Successful!
Now Login to continue.
===== Login =====
Username: ALIZA JAVED
Password: ASD123
Invalid Username!
alisha@kali: ~$ python3 secure_auth.py
===== Secure Authentication System =====
1. Register
2. Login
Enter your choice (1/2): 2
===== Login =====
Username: ALIZA JAVED
Password: ASD123
Login Successful!
alisha@kali: ~$ python3 secure_auth.py
===== Secure Authentication System =====
1. Register
2. Login
Enter your choice (1/2): 2
===== Login =====
Username: ALI
Password: asd123
Login Successful!
alisha@kali: ~$
```

Execution 3: I registered another user and, within the same execution, attempted to log in using the credentials of a previously registered user. The system correctly rejected the authentication, ensuring that users cannot access another user's account with incorrect credentials.

Final Execution: I entered the correct username and password, and the system authenticated the user successfully, confirming that the secure authentication system works as intended.

6. Results

The following table summarizes the outcomes of the implemented tasks.

Task	Status	Result
Password Hashing (bcrypt)	Completed	Passwords were securely hashed before storage.
Password Verification	Completed	Valid passwords were successfully verified.
SHA-256 Hashing	Completed	SHA-256 hashes were generated correctly.
Data Encryption	Completed	Messages were successfully encrypted using Fernet.
Data Decryption	Completed	Original messages were recovered using the correct secret key.
Wrong Key Handling	Completed	Invalid secret keys raised a decryption exception.
User Registration	Completed	New users were registered successfully.
User Login	Completed	Authentication succeeded for valid credentials.
User Data Storage	Completed	Hashed passwords were stored securely in <code>users.json</code> .

Result Summary

The project successfully demonstrated secure authentication and cryptographic techniques using Python. Passwords were protected through bcrypt hashing, sensitive data was encrypted and decrypted using Fernet, and a secure authentication system was implemented with user registration and login functionality. All required project objectives were achieved successfully.